

## 实验3 多线程程序设计

数据 231 235127 艾洋

**实验目的：**理解多线程的概念，掌握创建、管理和控制 Java 线程对象的方法，包括创建 Java 线程对象、改变线程状态、设置线程优先级及控制线程调度等方法，掌握实现线程互斥和线程同步的方法。

### 3-1

#### 实验内容：

1、编写一个有两个线程的程序，第一个线程用来计算 1~100 之间的偶数及个数，第二个线程用来计算 400-600 之间的奇数数及个数。

**【设计思路】：**采用继承 Thread 类的方式实现多线程，定义了两个线程类 MyThread1 和 MyThread2，其中 MyThread1 用来输出任意两个整数之间的偶数及个数，MyThread2 用来输出任意两个整数之间的奇数及个数。主函数中定义两个线程类的对象，然后通过 start()函数启动线程。

#### 【程序源代码】

```
import java.util.*;
public class Count extends Thread{
    /*public Count(){}*///构造不用写
    public void run(){
        for(int i=1;i<=100;i++){
            if(i%2==0)
                System.out.println(i+"是偶数");
        }
    }
}
class CountJi extends Thread{
    public void run(){
        for(int j=400;j<=600;j++){
            if(j%2!=0)
                System.out.println(j+"是奇数");
        }
    }
}
class Main{
    public static void main(String args[]){
        Count c=new Count();
```

```
        CountJi cj= new CountJi();
        c.start();
        cj.start();
    }
}
```

### 【试验结果】



The screenshot shows the output of a Java program in the IntelliJ IDEA console. The output consists of 20 lines, alternating between even and odd numbers. The first 10 lines are even numbers (2, 4, 6, 8, 10, 12, 14, 16, 18, 20) followed by 10 lines of odd numbers (401, 403, 405, 407, 409, 411, 413, 415, 417, 419). The console window title is 'Main' and the command line shows the Java executable path and the IntelliJ IDEA community edition version 2024.3.1.

```
运行 Main x
"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:D:\软件\IntelliJ IDEA Community Edition 2024.3.1.
2是偶数
4是偶数
6是偶数
8是偶数
10是偶数
12是偶数
14是偶数
16是偶数
18是偶数
20是偶数
401是奇数
403是奇数
405是奇数
407是奇数
409是奇数
411是奇数
413是奇数
415是奇数
417是奇数
419是奇数
```

### 【结果分析】

在本次实验中,通过创建两个线程类 Count 和 CountJi 来演示 Java 中线程的基本使用。Count 线程负责打印 1 到 100 之间的偶数,而 CountJi 线程负责打印 400 到 600 之间的奇数。主类 Main 通过调用这两个线程的 start 方法来启动它们。实验结果显示,两个线程能够并行运行,分别完成各自的任务,体现了多线程编程在并发执行任务上的优势。然而,由于 Java 线程调度的非确定性,两个线程的输出可能会交错在一起,这表明在多线程环境中需要考虑线程安全和同步问题,以确保程序的正确性和稳定性。

3-2

2、编写一个有两个线程的程序，第一个线程输出 1~100 之间的整数，第二个线程打印“HelloWorld”。

【设计思路】采用继承 Thread 类的方式实现多线程，定义了两个线程类 MyThread1 和 MyThread2，其中 MyThread1 用来输出任意两个数之间的整数，MyThread2 用来打印 HelloWorld。主函数中定义两个线程类的对象，然后通过 start()函数启动线程。

【程序源代码】

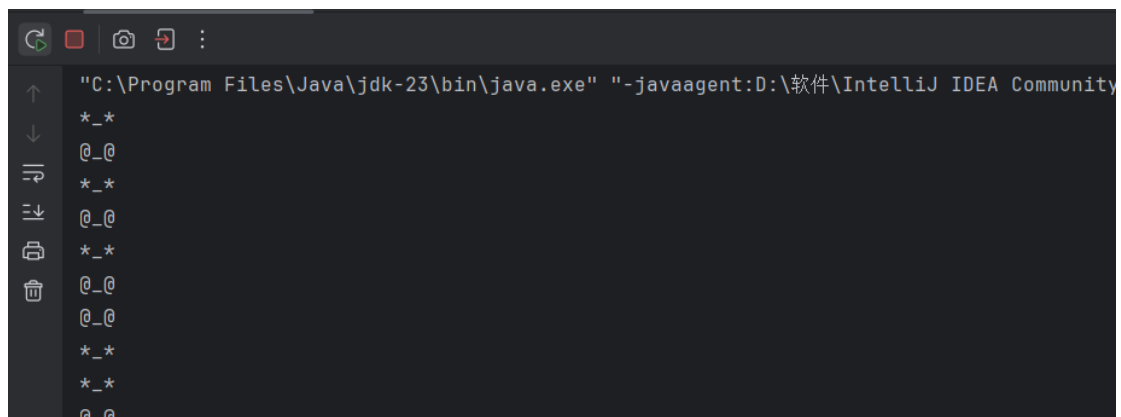
```
class Tortoise extends Thread
{ int sleepTime=0, liveLength=0;
  public Tortoise(String name,int sleepTime, int liveLength)
  {
    super(name);
    this.sleepTime=sleepTime;
    this.liveLength=liveLength;
  }
  public void run()
  { while (true )
    {
      liveLength--;
      System.out.println("@_@");
      try{
        sleep(sleepTime);
      }
      catch (InterruptedException e) {}
      if (liveLength<=0 )
      {
        System.out.println(getName()+"进入死亡状态\n");
        break;
      }
    }
  }
}
class Rabbit extends Thread
{
  int sleepTime=0, liveLength=0;
  public Rabbit(String name,int sleepTime, int liveLength)
  {
    super(name);
    this.sleepTime=sleepTime;
```

```

        this.liveLength=liveLength;
    }
    public void run()
    {
        while (true )
        {
            liveLength--;
            System.out.println("*_*");
            try{
                sleep( sleepTime);
            }
            catch (InterruptedException e) {}
            if (liveLength<=0 )
            {
                System.out.println(getName()+"进入死亡状态\n");
                break;
            }
        }
    }
}
}
public class ThreadExample
{
    public static void main(String a[])
    {
        Rabit rabbit= new Rabit("bai",100,100);// 新建线程 rabbit
        Tortoise tortoise = new Tortoise("gui",100,120);// 新建线程 tortoise
        rabbit.start();
        tortoise.start();
    }
}

```

### 【试验结果】



```

"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:D:\软件\IntelliJ IDEA Community
*_*
@_@
*_*
@_@
*_*
@_@
*_*
@_@
*_*
@_@

```

### 【结果分析】

在本次实验中，设计了两个线程类 `Tortoise` 和 `Rabit`，分别模拟乌龟和兔子的生命周期。每个线程对象在创建时被赋予特定的睡眠时间和生命周期长度。在 `run` 方法中，线程不断减少生命周期长度，并在每次循环中休眠指定的时间。当生命周期长度减至零或以下时，线程输出相应的消息并终止。主类 `ThreadExample` 创建并启动了这两个线程，展示了多线程环境下不同线程的并发执行。实验结果表明，尽管两个线程的睡眠时间相同，但由于线程调度的不确定性，它们的输出顺序和频率可能不同，这体现了多线程程序中线程执行的非确定性。此外，实验还展示了如何通过继承 `Thread` 类和重写 `run` 方法来创建和管理线程，为进一步探索线程同步和并发控制提供了基础。

**实验内容：**

编写一个 Java 应用程序，在主线程中创建三个线程：zhangWorker，wangWorker 和 boss。线程 zhangWorker 和 wangWorker 分别负责在命令行输出“搬运苹果”和“搬运香蕉”，这两个线程分别各自输出 20 行，每输出一行信息就准备休息 10 秒钟，但是 boss 线程负责不让 zhangWorker 和 wangWorker 休息。按模板要求，将【代码 1】~【代码 8】替换为 Java 程序代码。

**【设计思路】**

本实验为填空，按照样例来就可以

**【程序源代码】**

```
class Shop implements Runnable
{
    Thread zhangWorker, wangWorker, boss;
    public Shop()
    {
        boss = new Thread(this);
        zhangWorker = new Thread(this);
        wangWorker = new Thread(this);
        boss.setName("老板");
        zhangWorker.setName("张工");
        wangWorker.setName("王工");
    }
    public void run()
    {
        int i=0;
        if (Thread.currentThread()==zhangWorker)
        {
            while (true)
            {
                try { i++;
                    System.out.printf("\n%s 已搬运了%d 箱苹果\n",zhangWorker.getName(),i);
                    if (i==3)
                        return;
                    Thread.sleep(5000);
                }
                catch(InterruptedException e)
```

```

        {
            System.out.printf("\n%s 让 %s 继续工作",boss.getName(),zhangWorker.getName(),i);
        }
    }
}
else if (Thread.currentThread()==wangWorker)
{
    while (true )
    {
        try { i++;
            System.out.printf("\n%s 已搬运了%d 箱香蕉\n",wangWorker.getName(),i);
            if (i==3)
                return ;
            Thread.sleep(5000);
        }
        catch(InterruptedException e)
        {
            System.out.printf("\n%s 让 %s 继续工作",boss.getName(),wangWorker.getName(),i);
        }
    }
}
else if (Thread.currentThread()==boss)
{
    while (true)
    {
        synchronized (this) {
            notifyAll(); // 唤醒所有等待的工人
        }
        if (!(wangWorker.isAlive()|| zhangWorker.isAlive()))
        {
            System.out.printf("%n%s 下班",boss.getName());
            return;
        }
    }
}
}
}

public class ShopExample
{
    public static void main(String a[])
    {

```

```
        Shop shop=new Shop();
        shop.boss.start();
        shop.zhangWorker.start();
        shop.wangWorker.start();
    }
}
```

### 【试验结果】



```
运行 ShopExample x
"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:D:\软件\IntelliJ IDEA Community Edition 2024.3.1\
张工 已搬运了1 箱苹果
王工 已搬运了1 箱香蕉
张工 已搬运了2 箱苹果
王工 已搬运了2 箱香蕉
张工 已搬运了3 箱苹果
王工 已搬运了3 箱香蕉
老板 下班
进程已结束，退出代码为 0
```

### 【结果分析】

在本次实验中，通过实现 `Runnable` 接口创建了一个名为 `Shop` 的类，模拟了一个商店中老板和两名工人的工作流程。该程序中，老板线程负责监督张工和王工的工作，并在他们完成任务后通知他们下班；而张工和王工线程则分别负责搬运苹果和香蕉，每完成一次搬运后休眠 5 秒。实验结果显示，通过使用 `synchronized` 关键字和 `notifyAll` 方法，老板能够有效地管理和协调工人的工作，确保了线程间的同步和协作。此外，程序还展示了如何通过 `Thread.currentThread()` 方法来区分当前运行的线程，以及如何使用 `InterruptedException` 来处理线程被中断的情况。整体而言，该实验成功演示了多线程环境下线程的创建、管理和同步控制，为理解并发编程提供了一个实际的应用场景。